Pytex.school

От вайб-кодинга к профессиональной AI-разработке

День 1: Правила, скилы и инженерный харнес на Python-проекте linksvc

Алексей Жиряков
Pytex.school

ПРЕПОДАЮ Я, АЛЕКСЕЙ ЖИРЯКОВ, ТЕХ. РУКОВОДИТЕЛЬ В СБЕРБАНКЕ

15+ лет в backend-разработке, Техлид инженерных команд
Член программного комитета Highload

Сбербанк	Техлид	Отвечаю за работу с данными для ИИ на платформе ГенИИ
KION	CTO направления	Построил highload-архитектуру, платформу данных и персональные витрины
MTC Web Services	Техлид	Запустил AI-агентов для автотестов и ускорения код-ревью



ПЛАН ЭФИРА

01 Почему умный чат ломает продакшен: 3 боли

02 Инженерный харнес и стандарт SKILL.md

03 Вход в проект и закон репозитория AGENTS.md

04 Тестовая защита, миграции и безопасный рефакторинг

05 Итоги теории, 5 артефактов и переход к практике

ПОЧЕМУ УМНЫЙ ЧАТ ЛОМАЕТ ПРОДАКШЕН

3 ключевые боли вайб-кодинга на реальном сервисе linksvc

Мощная модель без детерминированного верификатора не лечит сбои, а генерирует их быстрее и убедительнее.

Вайб-кодинг создаёт иллюзию работающего сервиса, скрывая дефекты за зелёными локальными тестами.

3 ключевые боли вайб-кодинга: что ломается в linksvc

1. Боль 1. Бесконечный цикл правок (Fix Loop of Death)

   Агент рапортует «Всё готово!», но ходит по кругу, срезает assert и прячет сбои.

3 ключевые боли вайб-кодинга: что ломается в linksvc

1. Боль 1. Бесконечный цикл правок (Fix Loop of Death)

   Агент рапортует «Всё готово!», но ходит по кругу, срезает assert и прячет сбои.

2. Боль 2. Каскадный разрыв контракта (DTO)

   Самовольный рефакторинг полей ломает внешние сервисы и клиентов.

3 ключевые боли вайб-кодинга: что ломается в linksvc

1. Боль 1. Бесконечный цикл правок (Fix Loop of Death)

   Агент рапортует «Всё готово!», но ходит по кругу, срезает assert и прячет сбои.

2. Боль 2. Каскадный разрыв контракта (DTO)

   Самовольный рефакторинг полей ломает внешние сервисы и клиентов.

3. Боль 3. Архитектурный дрейф и слом конвенций

   Агент нарушает границы слоёв, тащит чужой стек и перекраивает чужой код.

Ловушка «среднего Python» (The Average Python Trap)

Синхронный вызов в async

`requests.get(url)`

Блокирует event loop FastAPI под нагрузкой.

Дата без часового пояса

`datetime.now()`

Наивное время ломает сравнение с UTC в базе.

Подавление исключений

```
except Exception: pass
```

Скрывает сбой БД и коллизии, клиент видит ложный 200 OK.

Модель повторяет средний открытый код. Отсюда и архитектурный дрейф: чужие библиотеки и стиль вместо конвенций команды.

Боль №1: упавший тест и реакция агента

✕ Чат-агент вырезает проверку

```
def test_create_link_bad_url():
    # Агент 'починил' упавший тест:
-   res = client.post('/links', json={'url':
'bad'})
-   assert res.status_code == 422
+   # assert временно закомментирован
+   pass  # тест теперь зелёный!

[!] Ложный успех: валидация схемы
    сломана, невалидный URL идёт в БД.
```

Боль №1: упавший тест и реакция агента

✕ Чат-агент вырезает проверку

```
def test_create_link_bad_url():
    # Агент 'починил' упавший тест:
-   res = client.post('/links', json={'url':
'bad'})
-   assert res.status_code == 422
+   # assert временно закомментирован
+   pass  # тест теперь зелёный!

[!] Ложный успех: валидация схемы
    сломана, невалидный URL идёт в БД.
```

✅ Инженерное лечение со скилом

```
class LinkCreate(BaseModel):
    # Чиним контракт схемы, а не тест!
+   original_url: HttpUrl

def test_create_link_bad_url(client):
    res = client.post('/links', json={'url':
'bad'})
    assert res.status_code == 422
    # ruff + basedpyright strict: exit 0
```

Боль №2: каскадный разрыв контракта

✕ Рефакторинг: скрытая дельта DTO

```
class LinkResponse(BaseModel):
    id: UUID
-   short_code: str  # исходный контракт
+   code: str  # 'сделал аккуратнее'

[!] Внешние потребители сломаны:
    - Worker: KeyError('short_code')
    - Mobile Client: 500 parse error
```

Боль №2: каскадный разрыв контракта

✕ Рефакторинг: скрытая дельта DTO

```
class LinkResponse(BaseModel):
    id: UUID
-   short_code: str  # исходный контракт
+   code: str  # 'сделал аккуратнее'

[!] Внешние потребители сломаны:
    - Worker: KeyError('short_code')
    - Mobile Client: 500 parse error
```

✅ Защита контрактов со скилом

```
class LinkResponse(BaseModel):
    id: UUID
    short_code: str  # НЕПРИКОСНОВЕННО
    original_url: HttpUrl

√ Спека API зафиксирована на диске
√ Тесты текущего поведения ловят дельту
√ Обратная совместимость: контракт API
зафиксирован
```

Боль №3: архитектурный дрейф и слом конвенций

✕ Хаос: слом слоёв и чужой стек

```
@router.post('/links/{code}/check')
async def check_link(code: str, db: Session):
    # 1. Нарушен слой: прямой SQL в роутере
    raw = db.execute(f'SELECT url FROM
links ... ')
    # 2. Чужой стек: синхронный вызов в async!
    res = requests.get(raw.scalar())
    # 3. Самовольный рефакторинг соседей:
    # переписал BaseService под свой вкус

[!] Event loop заблокирован на 5 секунд;
    Слои размыты, архитектура разрушена.
```

Боль №3: архитектурный дрейф и слом конвенций

✕ Хаос: слом слоёв и чужой стек

```
@router.post('/links/{code}/check')
async def check_link(code: str, db: Session):
    # 1. Нарушен слой: прямой SQL в роутере
    raw = db.execute(f'SELECT url FROM
links ... ')
    # 2. Чужой стек: синхронный вызов в async!
    res = requests.get(raw.scalar())
    # 3. Самовольный рефакторинг соседей:
    # переписал BaseService под свой вкус

[!] Event loop заблокирован на 5 секунд;
    Слои размыты, архитектура разрушена.
```

✅ Инженерный харнес: слои и бюджет

```
@router.post('/links/{code}/check')
async def check_link(
    code: str,
    svc: LinkService =
Depends(get_link_service),
):
    # Сервисный слой, async httpx, чистый роутер
    return await svc.check_availability(code)

√ Роутер не знает про БД (чистый HTTP)
√ Асинхронный httpx не блокирует event loop
√ Бюджет правок: строго ⩽ 5 файлов
```

Локальные тесты агента были зелёными, потому что он сам переписал их под свой баг.

Зелёный прогон внутри репозитория не защищает потребителей снаружи.

Лестница принуждения: от надежды на модель к детерминированному барьеру

Скил сам по себе — текст, а не замок

Вероятностная природа: модель срезает углы в длинном диалоге

Тестовый судья: агент переписывает assert под свой баг

Тавтологичные проверки: assert True вместо реальной логики

Выборочное послушание: пропуск стоп-условий ради 200 OK

Лестница принуждения: от надежды на модель к детерминированному барьеру

Скил сам по себе — текст, а не замок

Вероятностная природа: модель срезает углы в длинном диалоге

Тестовый судья: агент переписывает assert под свой баг

Тавтологичные проверки: assert True вместо реальной логики

Выборочное послушание: пропуск стоп-условий ради 200 OK

Внешнее детерминированное принуждение

Характеризационные тесты: текущее поведение зафиксировано до правок

PreToolUse хуки: физический запрет записи в tests/ и конфиги

Mutation Drill: тест обязан падать при контролируемой поломке

Серверный CI: ruff + basedpyright + pytest как детерминированный рубеж

ИНЖЕНЕРНЫЙ ХАРНЕС И СКИЛЫ

Как превратить вероятностную LLM в предсказуемого инженера

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

2. Слой 2. Процедурные скилы (SKILL.md)

   Повторяемые процедуры: фазы, критерии готовности, стоп-условия.

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

2. Слой 2. Процедурные скилы (SKILL.md)

   Повторяемые процедуры: фазы, критерии готовности, стоп-условия.

3. Слой 3. Машинные гардрайлы (ruff, basedpyright, pytest)

   Детерминированные ворота качества: нарушение не проходит в мастер.

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

2. Слой 2. Процедурные скилы (SKILL.md)

   Повторяемые процедуры: фазы, критерии готовности, стоп-условия.

3. Слой 3. Машинные гардрайлы (ruff, basedpyright, pytest)

   Детерминированные ворота качества: нарушение не проходит в мастер.

4. Слой 4. Детерминированные Quality Gates

   Коды возврата 0, characterization tests и независимый ревьюер.

AGENTS.md против SKILL.md: разделение ролей

AGENTS.md: Конституция правил

Роль: неизменяемые законы и границы дозволенного

Фокус: ЧТО ЗАПРЕЩЕНО (не трогать prod-схему, strict typing)

Жизненный цикл: всегда активен, нулевой шаг каждой сессии

Объем: компактный манифест (100-200 строк), навигация и запреты

AGENTS.md против SKILL.md: разделение ролей

AGENTS.md: Конституция правил

Роль: неизменяемые законы и границы дозволенного

Фокус: ЧТО ЗАПРЕЩЕНО (не трогать prod-схему, strict typing)

Жизненный цикл: всегда активен, нулевой шаг каждой сессии

Объем: компактный манифест (100-200 строк), навигация и запреты

SKILL.md: Алгоритмы достижения цели

Роль: пошаговая процедура решения инженерной задачи

Фокус: КАК СДЕЛАТЬ (фазы, чеклисты, стоп-условия, откат)

Жизненный цикл: по требованию (on-demand progressive disclosure)

Объем: рабочий стандарт ≤400 строк (спецификация до 500 строк)

Реальный AGENTS.md из Git: архитектура и Anti-Cheat

fastapi/.agents/AGENTS.md: Границы слоёв

```
# Архитектурные границы слоёв сервиса:
# 1. app/api/v1/endpoints/ — только приём HTTP
#    и валидация через Pydantic DTO
(app/schemas/)
-   from sqlalchemy.ext.asyncio import
AsyncSession
+   # ЗАПРЕТ: прямой SQL или сессия в роутере!
# 2. app/services/ — бизнес-логика и транзакции
-   from fastapi import HTTPException
+   # ЗАПРЕТ: HTTPException в сервисе (только
домен)
# 3. app/repositories/ — изоляция запросов к БД
```

Реальный AGENTS.md из Git: архитектура и Anti-Cheat

fastapi/.agents/AGENTS.md: Границы слоёв

```
# Архитектурные границы слоёв сервиса:
# 1. app/api/v1/endpoints/ — только приём HTTP
#    и валидация через Pydantic DTO
(app/schemas/)
-   from sqlalchemy.ext.asyncio import
AsyncSession
+   # ЗАПРЕТ: прямой SQL или сессия в роутере!
# 2. app/services/ — бизнес-логика и транзакции
-   from fastapi import HTTPException
+   # ЗАПРЕТ: HTTPException в сервисе (только
домен)
# 3. app/repositories/ — изоляция запросов к БД
```

fastapi/.agents/AGENTS.md: Anti-Cheat запреты

```
# Строгие запреты против обхода проверок:
-   except Exception: pass  # СТРОГО ЗАПРЕЩЕНО!
+   raise ShortCodeAlreadyExistsError(code)
-   # assert res.status_code == 422  # ЗАПРЕЩЕНО
слабить
+   assert res.status_code == 422  # строгий чек
-   fail_under = 75  # ЗАПРЕЩЕНО снижать
покрытие!
+   fail_under = 85  # правило: только повышать

√ Ворота DoD: ruff + pyright + pytest == 0
```

Реальный SKILL.md из Git: послойная разработка фичи

fastapi/.agents/skills/fastapi-feature (L1)

```
---
name: fastapi-feature
description: >-
  Послойно разрабатывает новую функциональность
  FastAPI (схемы DTO → репозиторий → сервис
  → роутер) по принципу Red-Green-Refactor
  и защищает старые контракты.
---

# L1 Frontmatter: ~100 токенов в системном
индексе.
# Тело скила не сжигает контекст до вызова!
```

Реальный SKILL.md из Git: послойная разработка фичи

fastapi/.agents/skills/fastapi-feature (L1)

```
---
name: fastapi-feature
description: >-
  Послойно разрабатывает новую функциональность
  FastAPI (схемы DTO → репозиторий → сервис
  → роутер) по принципу Red-Green-Refactor
  и защищает старые контракты.
---

# L1 Frontmatter: ~100 токенов в системном
индексе.
# Тело скила не сжигает контекст до вызова!
```

Пошаговый алгоритм и ворота DoD (L2)

```
### Фаза 1: Защитный слепок
tests/characterization/
### Фаза 2: DTO схемы с from_attributes=True
### Фаза 3: Красный тест (Red-Green-Refactor)
### Фаза 4: Реализация снизу вверх:
  1. repositories/ → 2. services/ → 3.
endpoints/
### Фаза 5: Ворота Definition of Done:
  1. uv run ruff check && ruff format --check ==
0
  2. uv run basedpyright == 0
  3. uv run pytest --cov-fail-under=85 == 0
[!] СТОП: падение слепка → git restore app/
```

Каталог скилов проекта linksvc: 17 процедур в Git

Разработка и рефакторинг

`fastapi-feature`, `safe-refactoring`
Послойная реализация эндпоинтов и безопасный рефакторинг с сохранением контрактов.

Тестовая защита

`characterization-tests`, `tests-unit`
Защитный слепок существующего поведения, честные unit и integration тесты на живой БД.

База данных и миграции

`database-migration`

Двунаправленные миграции Alembic по принципу маятника с обязательным downgrade().

Скилы лежат в .agents/skills/ и связаны через .opencode/skills -> ../.agents/skills. OpenCode активирует их нативно по имени.

Почему нельзя всё сложить в один большой AGENTS.md?

Ловушка мега-файла (Mega-Prompt Trap)

- Угар контекста: 3000 строк = 30k токенов на каждый шаг
- Constraint Fatigue: модель игнорирует правила в списке
- Lost in the Middle: середина файла вымывается первой
- Информационный шум: чужие правила мешают текущей задаче

Почему нельзя всё сложить в один большой AGENTS.md?

Ловушка мега-файла (Mega-Prompt Trap)

Угар контекста: 3000 строк = 30k токенов на каждый шаг

Constraint Fatigue: модель игнорирует правила в списке

Lost in the Middle: середина файла вымывается первой

Информационный шум: чужие правила мешают текущей задаче

Архитектура Progressive Disclosure

Каталог frontmatter: ~100 токенов в системном индексе

Точечная загрузка: тело скила читается строго под задачу

Экономия контекста: процедуры вынесены из системного промпта

Изоляция знаний: каждый скил автономен и тестируется отдельно

Спецификация Agent Skills: происхождение стандарта

Хронология стандарта (окт 2025 — 2026)

16 октября 2025: Anthropic публикует черновик стандарта

18 декабря 2025: Альянс открывает agentskills.io

Начало 2026: кросс-платформенная поддержка во всех средах

Принято индустрией: Claude Code, OpenCode, Cursor, Droid, AGY

Спецификация Agent Skills: происхождение стандарта

Хронология стандарта (окт 2025 — 2026)

16 октября 2025: Anthropic публикует черновик стандарта

18 декабря 2025: Альянс открывает agentskills.io

Начало 2026: кросс-платформенная поддержка во всех средах

Принято индустрией: Claude Code, OpenCode, Cursor, Droid, AGY

3 уровня спецификации Agent Skills

L1 Frontmatter: YAML-метаданные (~100 токенов в индексе)

L2 Procedural Body: файл SKILL.md ≤500 строк (норма ≤400)

L3 Supporting Resources: scripts/, references/ по ссылке

Вендор-нейтральность: 0 внешних облачных зависимостей

Каркас Agent Skills: файловая структура и уровни L1–L3

Файловая структура каркаса в Git

```
.agents/skills/fastapi-feature/
├── SKILL.md              # L1 + L2
│   ├── frontmatter        # L1 (~100 tok)
│   └── procedural body    # L2 (≤400 строк)
├── scripts/              # L3 Ресурсы
│   └── verify.sh          # Быстрый валидатор
└── references/           # L3 Справочники
    └── contracts.md       # Data-контракты

√ Автономен: 0 внешних облачных зависимостей
```

Каркас Agent Skills: файловая структура и уровни L1–L3

Файловая структура каркаса в Git

```
.agents/skills/fastapi-feature/
├── SKILL.md              # L1 + L2
│   ├── frontmatter        # L1 (~100 tok)
│   └── procedural body    # L2 (≤400 строк)
├── scripts/              # L3 Ресурсы
│   └── verify.sh          # Быстрый валидатор
└── references/           # L3 Справочники
    └── contracts.md       # Data-контракты
```

√ Автономен: 0 внешних облачных зависимостей

Связка уровней: от каталога к исполнению

```
# 1. L1 Frontmatter: системный каталог
#    Имя + описание = ~100 токенов в индексе
# 2. L2 Procedural Body: фазы и DoD
#    Загружается точечно под конкретную задачу
# 3. L3 Supporting Resources: по ссылке
#    Скрипты и схемы читаются строго по нужде
```

√ Чистый контекст: 0 информационного шума
√ Вендор-нейтрально: OpenCode, Cursor, Claude

Анатомия YAML Frontmatter: правила name и description

Поле name: идентификатор процедуры

```
---
name: fastapi-feature
---

# 1. Строгий kebab-case: только латиница и дефис
# 2. Правило 1:1: имя совпадает с
папкой .agents/skills/name/
# 3. Лимит длины: ≤64 символа, глагол + объект
# 4. Уникальность: глобальный ключ в каталоге
репозитория

√ Без пробелов, подчеркиваний и спецсимволов
```

Анатомия YAML Frontmatter: правила name и description

Поле name: идентификатор процедуры

```
---
name: fastapi-feature
---

# 1. Строгий kebab-case: только латиница и дефис
# 2. Правило 1:1: имя совпадает с
папкой .agents/skills/name/
# 3. Лимит длины: ≤64 символа, глагол + объект
# 4. Уникальность: глобальный ключ в каталоге
репозитория

√ Без пробелов, подчеркиваний и спецсимволов
```

Поле description: семантический маршрутизатор

```
description: >-
  Послойно реализует новую фичу FastAPI.
  КОГДА ПРИМЕНЯТЬ: новые эндпоинты, DTO,
сервисы.
  НЕ ПРИМЕНЯТЬ: для багфикса (bugfix),
  рефакторинга (safe-refactoring) или схем БД.
---

# 1. Семантический роутинг: модель выбирает скил
только по нему
# 2. Паттерн двух списков: When to use vs When
NOT to use
# 3. Бюджет: ≤100 токенов (не пересказывать
алгоритм!)
```

Альтернатива AWS: Bedrock Agents и облачный рантайм

Суть архитектуры AWS Bedrock Agents

Облачный SaaS: конфигурация агента в веб-консоли AWS

Action Groups: интеграция с кодом через вызовы AWS Lambda

Knowledge Bases: автоматический RAG через OpenSearch и S3

Безопасность: доступ через роли IAM и Guardrails for Bedrock

Плюс: готовая инфраструктура AWS и аудит CloudWatch

Альтернатива AWS: Bedrock Agents и облачный рантайм

Суть архитектуры AWS Bedrock Agents

Облачный SaaS: конфигурация агента в веб-консоли AWS

Action Groups: интеграция с кодом через вызовы AWS Lambda

Knowledge Bases: автоматический RAG через OpenSearch и S3

Безопасность: доступ через роли IAM и Guardrails for Bedrock

Плюс: готовая инфраструктура AWS и аудит CloudWatch

Почему не подходит для Git-контура разработки

Оторванность от кода: промпты живут в консоли, а не в Git-ветке

Нельзя запустить локально: тесты требуют живого облака AWS

Инерция деплоя: правка правила требует обновления Action Group

Жёсткий Vendor Lock-in: зависимость от закрытого API AWS

Изоляция от IDE: Cursor, OpenCode, Claude Code не видят агента

Альтернатива Microsoft: Semantic Kernel и Heavy SDK

Суть архитектуры Semantic Kernel / AutoGen

Code-First SDK: библиотека встраивается в бэкенд (C#, Python)

Native Plugins: методы с декораторами @kernel_function

Встроенные коннекторы памяти к векторным БД (Qdrant, Pinecone)

AutoGen Multi-Agent: диалоговые агенты с шиной сообщений

Плюс: глубокая интеграция с кодом приложения и строгие типы

Альтернатива Microsoft: Semantic Kernel и Heavy SDK

Суть архитектуры Semantic Kernel / AutoGen

Code-First SDK: библиотека встраивается в бэкенд (C#, Python)

Native Plugins: методы с декораторами @kernel_function

Встроенные коннекторы памяти к векторным БД (Qdrant, Pinecone)

AutoGen Multi-Agent: диалоговые агенты с шиной сообщений

Плюс: глубокая интеграция с кодом приложения и строгие типы

Почему не подходит для управления AI-кодингом

Засорение продакшена: тяжелый рантайм попадает в зависимости

Перекомпиляция на каждую правку: сменил правило — деплой

Оторванность от AI-IDE: SDK исполняется внутри приложения

Хрупкость версий: частые мажорные ломающие изменения SDK

Git-скилы проще: чистый Markdown без сторонних библиотек

Скил — это не внешняя платформа и не SDK: это исполняемый регламент прямо в Git.

Он живёт в ветке фичи, проходит через Pull Request и исполняется любой моделью.

Сравнение подходов: чат, правила и SKILL.md

Критерий	Голый чат	Правила (Rules)	SKILL.md
Детерминизм процедуры	Реакция случайная, импровизация модели	Нет структуры шагов, зависит от формулировки	Стоп-условия и возврат к фазе скила

Сравнение подходов: чат, правила и SKILL.md

Критерий	Голый чат	Правила (Rules)	SKILL.md
Детерминизм процедуры	Реакция случайная, импровизация модели	Нет структуры шагов, зависит от формулировки	Стоп-условия и возврат к фазе скила
Защита от контекстного выгорания	Правила тонут в длинной истории	Всегда активны, занимают контекст каждой сессии	Три уровня: каталог, тело, ресурсы

Сравнение подходов: чат, правила и SKILL.md

Критерий	Голый чат	Правила (Rules)	SKILL.md
Детерминизм процедуры	Реакция случайная, импровизация модели	Нет структуры шагов, зависит от формулировки	Стоп-условия и возврат к фазе скила
Защита от контекстного выгорания	Правила тонут в длинной истории	Всегда активны, занимают контекст каждой сессии	Три уровня: каталог, тело, ресурсы
Переносимость между агентами и IDE	Нулевая: живёт в истории чата	Привязка к одной среде, например Cursor	OpenCode, Cursor, Copilot, Droid, Claude Code

Проектирование скилов: лимиты размера и декомпозиция

Нормы размера (≤400 строк)

Спецификация Agent Skills: индустриальная норма ≤500 строк

Защита от Constraint Fatigue (утомляемости ограничений)

Буфер безопасности (Margin of Safety): рабочий гейт ≤400 строк

Зоны светофора: ≤400 норма, 401–500 техдолг, >500 блокер

Проектирование скилов: лимиты размера и декомпозиция

Нормы размера (≤400 строк)

Спецификация Agent Skills: индустриальная норма ≤500 строк

Защита от Constraint Fatigue (утомляемости ограничений)

Буфер безопасности (Margin of Safety): рабочий гейт ≤400 строк

Зоны светофора: ≤400 норма, 401–500 техдолг, >500 блокер

Декомпозиция монолитов (One Job)

Антипаттерн: мега-скил на 2000 строк ломает контекст

Принцип One Job: одна процедура на одну задачу

Библиотека Pytex: 7 атомарных скилов конвейера

Каноны не копируем, ссылаемся на канонический путь

Декомпозиция скилов: разбиение монолита и перекрёстные ссылки

Разбиение монолита: принцип One Job

```
# Антипаттерн: Монолит 1500+ строк (ломает
контекст)
.agents/skills/mega-feature/SKILL.md ❌
├── Миграция Alembic (схемы, downgrade)
├── FastAPI эндпоинты (DTO, сервис, роуты)
└── Юнит & интеграционные тесты

# ✅ Декомпозиция: 3 атомарных скила (≤350
строк):
.agents/skills/
├── database-migration/SKILL.md  # Только БД
├── fastapi-feature/SKILL.md     # Только фича
└── tests-unit/SKILL.md          # Только тесты

√ Изоляция: 1 скил = 1 фаза и свои Definition of
Done
```

Декомпозиция скилов: разбиение монолита и перекрёстные ссылки

Разбиение монолита: принцип One Job

```
# Антипаттерн: Монолит 1500+ строк (ломает контекст)
.agents/skills/mega-feature/SKILL.md ❌
├── Миграция Alembic (схемы, downgrade)
├── FastAPI эндпоинты (DTO, сервис, роуты)
└── Юнит & интеграционные тесты

# ✅ Декомпозиция: 3 атомарных скила (≤350 строк):
.agents/skills/
├── database-migration/SKILL.md  # Только БД
├── fastapi-feature/SKILL.md     # Только фича
└── tests-unit/SKILL.md          # Только тесты

√ Изоляция: 1 скил = 1 фаза и свои Definition of Done
```

Как ссылаться: относительные пути в Git

```
# fastapi-feature/SKILL.md (фрагмент вызова)
## 3. Пошаговая процедура
### Шаг 1: Изменение схемы данных
Если фича требует новой таблицы или полей:
  1. ❌ НЕ писать миграцию вручную здесь!
  2. ✅ Открыть скил по каноническому пути:
     👉 ../database-migration/SKILL.md
  3. Выполнить проверку маятника (up/down)

### Шаг 4: Покрытие тестами
  - Открыть скил ../tests-unit/SKILL.md
  - Оформить тесты по канону FakeRepository

√ Каноны не копируем: единый источник правды в Git
```

ВХОД В СУЩЕСТВУЮЩИЙ ПРОЕКТ

3 шага: тесты поведения, база знаний по Карпати и скилы-гардрайлы

3 шага безопасного входа в существующий проект

Шаг 1. Защитные тесты

`тесты без правок кода`

Покрываем тестами текущего поведения всё, что не покрыто, НЕ ПЕРЕПИСЫВАЯ существующий код сервиса.

Шаг 2. База знаний по Карпати

`data-контракты & схемы`

Формируем контекст проекта: data-контракты DTO, схемы БД, архитектурные инварианты слоёв.

Шаг 3. Скилы и гардрайлы

`skills & quality gates`

Подключаем процедурные скилы и машинные гардрайлы (ruff, basedpyright, pytest) в CI/CD.

Безопасный вход: запечатываем статус-кво, строим базу знаний и ставим гардрайлы.

Шаг 1: Тесты текущего поведения (Characterization)

Принцип: фиксация статуса-кво

```
# Главный закон безопасного входа в legacy:
# Код сервиса ЗАПРЕЩЕНО менять или рефакторить
# до тех пор, пока поведение не запечатано!

√ Покрываем 'как есть': статус-коды, payload,
headers
√ Фиксируем де-факто контракт старых эндпоинтов
√ Защита от скрытых регрессий при будущих
правках
[!] Ошибка вайб-кодинга: сразу бросаться
рефакторить
```

Шаг 1: Тесты текущего поведения (Characterization)

Принцип: фиксация статуса-кво

```
# Главный закон безопасного входа в legacy:
# Код сервиса ЗАПРЕЩЕНО менять или рефакторить
# до тех пор, пока поведение не запечатано!

√ Покрываем 'как есть': статус-коды, payload, headers
√ Фиксируем де-факто контракт старых эндпоинтов
√ Защита от скрытых регрессий при будущих правках
[!] Ошибка вайб-кодинга: сразу бросаться рефакторить
```

tests/characterization/test_critical_path.py

```
@pytest.mark.asyncio
async def test_char_redirect_code_is_307(client):
    # Фиксируем статус-кво: редирект ОБЯЗАН быть 307!
    payload = {'original_url': 'https://pytex.ru', 'custom_code': 'c307'}
    await client.post('/api/v1/links', json=payload)
    res = await client.get('/api/v1/links/c307', follow_redirects=False)
    assert res.status_code == 307  # НЕ 301! Браузер не кэширует
    assert res.headers['location'] == 'https://pytex.ru'
    # Исходный код сервиса НЕ тронут ни на одну строчку!
```

Шаг 2: База знаний по Карпати (Data-контракты и инварианты)

Концепция Карпати: Context-First

```
# Репозиторий как база знаний для агента:
├── architecture-map.md  # Границы слоёв и
вызовы
├── data-contracts.md    # Спецификации DTO схем
└── db-schema.md         # Таблицы, индексы,
связи

√ Data-контракты: агент опирается на строгие
Pydantic
  схемы вместо догадок и галлюцинаций
√ Инварианты слоёв: правила вызовов
зафиксированы
```

Шаг 2: База знаний по Карпати (Data-контракты и инварианты)

Концепция Карпати: Context-First

```
# Репозиторий как база знаний для агента:
├── architecture-map.md  # Границы слоёв и вызовы
├── data-contracts.md    # Спецификации DTO схем
└── db-schema.md         # Таблицы, индексы, связи

√ Data-контракты: агент опирается на строгие Pydantic
  схемы вместо догадок и галлюцинаций
√ Инварианты слоёв: правила вызовов зафиксированы
```

app/schemas/link.py: Защищённый контракт DTO

```
class LinkResponse(BaseModel):
    # Строгий Data-контракт ответа сервиса linksvc:
    model_config = ConfigDict(from_attributes=True)
    id: UUID
    short_code: str  # ВНЕШНИЙ КОНТРАКТ (НЕ ТРОГАТЬ!)
    original_url: HttpUrl
    created_at: datetime

√ Спека API зафиксирована: агент не может переименовать
  поле short_code в code и сломать клиентов!
```

Шаг 3: Процедурные скилы и машинные гардрайлы

Процедурные скилы под задачи проекта

```
# Набор процедурных регламентов в
.agents/skills/:
- fastapi-feature: послойная разработка
эндпоинтов
- safe-refactoring: правки с контролем слепка
- database-migration: миграции Alembic (маятник)

√ Каждая задача выполняется по выверенному
алгоритму
√ При сбое слепка — автоматический откат git
restore
```

Шаг 3: Процедурные скилы и машинные гардрайлы

Процедурные скилы под задачи проекта

```
# Набор процедурных регламентов в
.agents/skills/:
- fastapi-feature: послойная разработка
эндпоинтов
- safe-refactoring: правки с контролем слепка
- database-migration: миграции Alembic (маятник)

√ Каждая задача выполняется по выверенному
алгоритму
√ При сбое слепка — автоматический откат git
restore
```

pyproject.toml: Детерминированные Quality Gates

```
[tool.basedpyright]
typeCheckingMode = 'strict'  # Строгая типизация
[tool.coverage.report]
fail_under = 85  # Покрытие тестами: только
вверх!

# Три кода возврата 0 подряд в pre-commit и CI:
√ uv run ruff check .
√ uv run basedpyright
√ uv run pytest -q
```

Вход в существующий проект — это не рефакторинг с первого дня.

Сначала запечатываем поведение тестами, затем формируем базу знаний и окружаем проект машинными гардрайлами.

ЗАКОН РЕПОЗИТОРИЯ И ВОРОТА КАЧЕСТВА

AGENTS.md, строгие запреты Anti-Cheat и правило «только вверх»

4 нерушимых правила AGENTS.md

1. Правило 1. Границы слоёв

   Роутер не ходит в БД, сервис не знает про HTTP.

4 нерушимых правила AGENTS.md

1. ### Правило 1. Границы слоёв

   Роутер не ходит в БД, сервис не знает про HTTP.

2. ### Правило 2. Транзакции и пул

   Сессия только через `Depends(get_db)`, `commit` в сервисе.

4 нерушимых правила AGENTS.md

1. ## Правило 1. Границы слоёв

   Роутер не ходит в БД, сервис не знает про HTTP.

2. ## Правило 2. Транзакции и пул

   Сессия только через `Depends(get_db)`, `commit` в сервисе.

3. ## Правило 3. Строгие запреты (Anti-Cheat)

   Запрещено подавление исключений, проглатывание ошибок, модификация тестов и `except Exception: pass`.

4 нерушимых правила AGENTS.md

1. **Правило 1. Границы слоёв**

   Роутер не ходит в БД, сервис не знает про HTTP.

2. **Правило 2. Транзакции и пул**

   Сессия только через `Depends(get_db)`, `commit` в сервисе.

3. **Правило 3. Строгие запреты (Anti-Cheat)**

   Запрещено подавление исключений, проглатывание ошибок, модификация тестов и `except Exception: pass`

4. **Правило 4. Правило «только вверх»**

   Порог coverage в `pyproject.toml` можно повышать, снижать нельзя.

Детерминированные ворота качества и гардрайлы

pyproject.toml: ruff, basedpyright, pytest-cov

```
[tool.ruff]
line-length = 100

[tool.basedpyright]
typeCheckingMode = "strict"

[tool.pytest.ini_options]
addopts = "-ra -q --strict-markers --cov"

[tool.coverage.report]
fail_under = 85
```

Детерминированные ворота качества и гардрайлы

pyproject.toml: ruff, basedpyright, pytest-cov

```
[tool.ruff]
line-length = 100

[tool.basedpyright]
typeCheckingMode = "strict"

[tool.pytest.ini_options]
addopts = "-ra -q --strict-markers --cov"

[tool.coverage.report]
fail_under = 85
```

Детерминированный верификатор

```
uv run ruff check .
uv run basedpyright
uv run pytest -q
```

Три кода возврата 0 подряд, иначе остановка.

Анатомия контракта SKILL.md: 6 секций

1. Назначение

Конкретная инженерная задача и гарантии результата.

2. Предусловия

Окружение: ветка, чистый git status, база данных.

3. Шаги процедуры

Детерминированные фазы выполнения в строгом порядке.

4. Артефакты

Файлы, diff, логи и handoff-пакет без лишнего мусора.

5. Приёмка и DoD

Машинные ворота качества: коды 0, тесты, линтеры.

6. Стоп-условия

Точки немедленной остановки, эскалация и откат.

Каждый скил — это самодостаточный исполняемый контракт с детерминированным DoD.

ТРЁХУРОВНЕВАЯ СИСТЕМА ТЕСТИРОВАНИЯ И ГАРДРАЙЛЫ

Snapshot-тесты API, покрытие критических ветвлений и интеграционные тесты с PostgreSQL

Пирамида тестового харнеса

1. Уровень 1. Characterization Tests

   Тесты текущего поведения до первой правки.

Пирамида тестового харнеса

1. Уровень 1. Characterization Tests

   Тесты текущего поведения до первой правки.

2. Уровень 2. Unit-тесты

   Тестирование доменных ветвлений на Fake-репозитории.

Пирамида тестового харнеса

1. Уровень 1. Characterization Tests

   Тесты текущего поведения до первой правки.

2. Уровень 2. Unit-тесты

   Тестирование доменных ветвлений на Fake-репозитории.

3. Уровень 3. Integration-тесты

   Реальная PostgreSQL и откат транзакции после каждого теста.

Тесты текущего поведения (Characterization)

✅ Контракт зафиксирован тестами до первой правки

```
async def test_characterization_redirect(client,
link):
    res = await
client.get(f'/{link.short_code}')
    assert res.status_code == 307
    assert res.headers['location'] == link.url
```

√ Зафиксирован контракт статуса 307
√ Тесты не зависят от моков сервиса

Тесты текущего поведения (Characterization)

✅ Контракт зафиксирован тестами до первой правки

```
async def test_characterization_redirect(client, link):
    res = await client.get(f'/{link.short_code}')
    assert res.status_code == 307
    assert res.headers['location'] == link.url
```

√ Зафиксирован контракт статуса 307
√ Тесты не зависят от моков сервиса

✕ Мгновенный сигнал поломки в CI

```
# Агент изменил код редиректа на 302:
>   assert res.status_code == 307
E   AssertionError: assert 302 == 307
E   + where 302 = <Response [302 Found]>

[!] Провал зафиксирован автоматически.
    Дефект отсечён до попадания в мастер.
    Агент обязан вернуть контракт 307.
```

Тесты текущего поведения защищают клиентов лучше сотни регламентов: машина ловит то, что пропускает глаз.

Любое изменение имени поля или кода ответа делает тест красным.

ФИЧА, МИГРАЦИЯ И РЕФАКТОРИНГ

Безопасный цикл изменений на примере expires_at и HTTP 410 Gone

Задача expires_at: вайб-кодинг и скилы

[!] Вайб-кодинг

Обязательное поле `expires_at`

Старые клиенты получают 422

Миграция без рабочего `downgrade()`

Нет ответа 410 Gone

[+] Скилы

`expires_at: datetime | None`

Старые ссылки работают как раньше

Истёкшая ссылка отдаёт 410 Gone

Миграция обратима

Реализация в Pydantic и SQLAlchemy

app/schemas/link.py и app/models/link.py

```
class LinkCreate(BaseModel):
    original_url: HttpUrl
    expires_at: datetime | None = None

    @field_validator("expires_at")
    @classmethod
    def must_have_tz(cls, v: datetime | None) → datetime | None:
        if v is not None and v.utcoffset() is None:
            raise ValueError("нужен часовой пояс, например UTC")
        return v

expires_at: Mapped[datetime | None] = mapped_column(
    DateTime(timezone=True), nullable=True)
```

Проверка обратимости миграции Alembic: принцип маятника

Принцип обратимости

Правило маятника: любая миграция схемы обязана быть двунаправленной.

Инвариант: агент обязан реализовать и проверить `downgrade()`. Если откат не работает — миграция считается дефектной.

```
uv run alembic upgrade head

uv run alembic downgrade -1

uv run alembic upgrade head
```

Контролируемая поломка (Mutation Drill)

Шаг 1. Внести поломку

```
now > expires_at
```

Контролируемый дефект в доменной логике.

Шаг 2. Запустить тесты

```
uv run pytest -q
```

Ожидаем красный результат: тест поймал поломку.

Шаг 3. Вернуть код

```
git restore
```

Тест, который остался зелёным, бесполезен: усиливаем его.

Тест, который не падает при поломке бизнес-логики, бесполезен.

Рефакторинг без тестов поведения - лотерея, с ними - безопасная процедура.

Меняем структуру, а не поведение: тесты поведения и мутационная проверка ловят регрессии.

ИТОГИ ЛЕКЦИИ И 5 АРТЕФАКТОВ

Чек-лист внедрения архитектурного харнеса в репозиторий

5 ключевых выводов лекции

1. Детерминированная верификация: решает код возврата машины, а не убеждения модели
2. AGENTS.md: закон репозитория: слои, нерушимые правила, только вверх
3. SKILL.md: процедура с критериями DoD и гардрайлами
4. Тесты текущего поведения: защита data-контрактов API до первой правки кода
5. Ворота качества и гардрайлы: ruff, basedpyright и pytest без исключений

5 артефактов в вашем репозитории linksvc

`AGENTS.md`	закон репозитория и запреты Anti-Cheat
`architecture-map.md`	карта слоёв и границы data-контрактов
`pyproject.toml`	ворота и гардрайлы: ruff, basedpyright, coverage
`.agents/skills`	7 процедурных скилов SKILL.md
`tests/`	тесты текущего поведения, unit и integration тесты

ОБСУДИМ ВАШИ ВОПРОСЫ

Архитектурный харнес, правила и скилы в ваших проектах